BİL4010

Paralel programlama

7.Hafta

MPI Trapezoidal Kurali

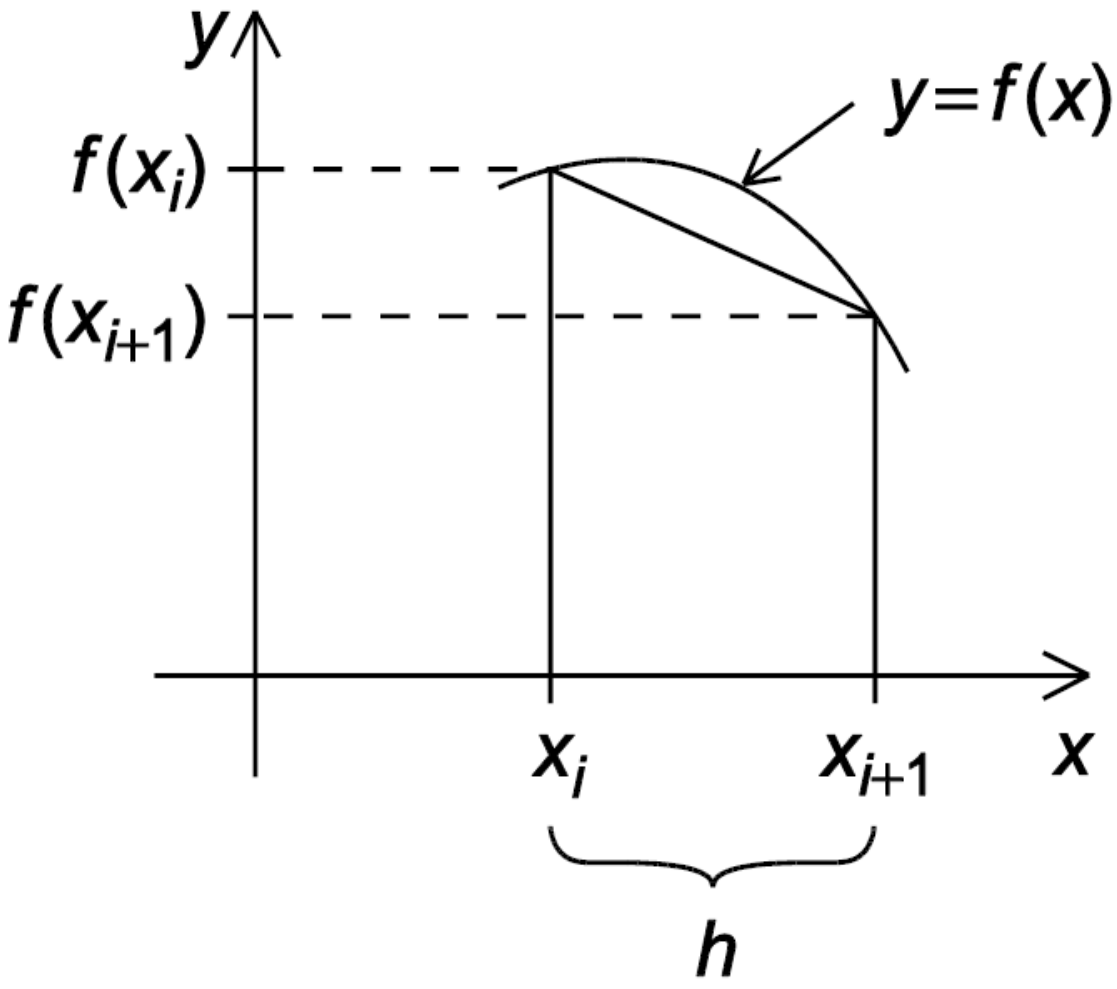
İçerik

- MPI ile Trapezoidal kuralının paralel olarak çalıştırılması
  - Trapezoid kuralı ile parçalı integral ile alan hesabı örneği
- MPI ile I/O kontrolü
- MPI ile kullanıcıdan girdi almak
- MPI ile hesaplama süresi denetimi

Trapezoidal (Yamuk) Kuralı

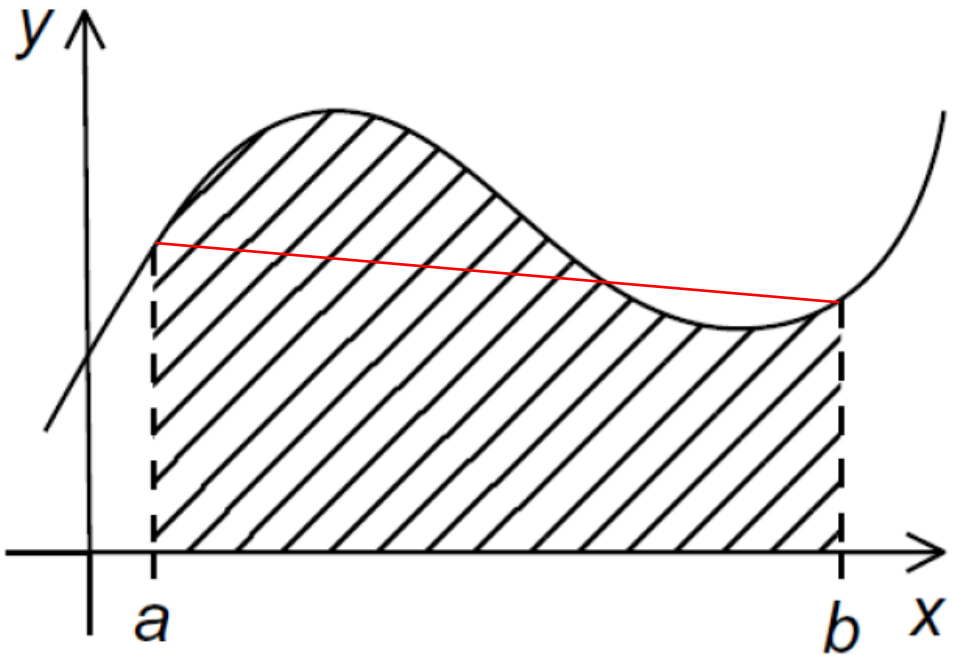
- Şimdiye kadar MPI ile işlemler arası haberleşmeyi ve mesaj gönderme ve alma fonksiyonları üzerinde durduk
- Artık biraz daha MPI ile nasıl paralel programlar gerçekleştirilir sorusunun cevabını örnekler ile inceleyelim
- Trapezoidal kuralı nümerik olarak integralin hesaplanması için kullanılan yöntemlerden bir tanesidir.
- MPI kullanarak bu yöntemi paralel olarak gerçekleştirelim.

Bir Trapezoid için Alan Hesabı

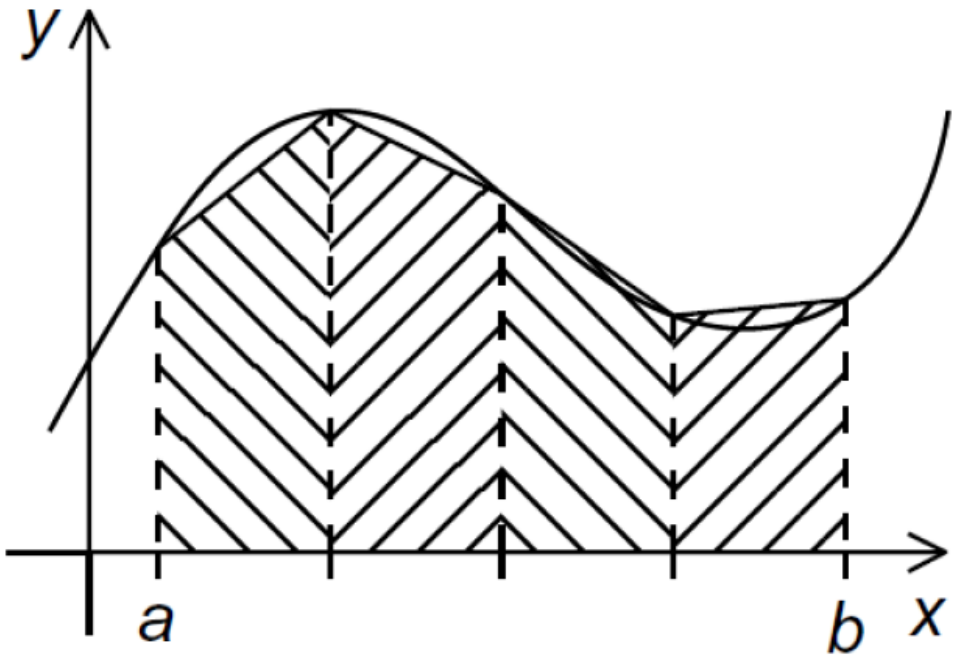


Yamuk Alanı:
(f(xi+1) + f(xi)) * (xi+1-xi) /2

Trapezoidal (Yamuk) Kuralı



(a)
Gerçek Alan



(b)
Trapezoid Kullanılarak Hesaplanan Yaklaşık Alan

Trapezoidal (Yamuk) Kuralı

$$\textit{Bir Trapezoidin Alanı} = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

Fonksiyonun altında kalan alanı n adet eşit parçaya böldüğümüzde her birinin yüksekliği

$$h = \frac{b-a}{n}$$

$$x_0 = a,\ x_1 = a+h,\ x_2 = a+2h,\ \ldots,\ x_{n-1} = a+(n-1)h,\ x_n = b$$

x eksenini n adet eşit parçaya bölerek alanı hesapladığımızda:

$$\int_{X_0}^{X_n} f(x)dx = \frac{h}{2}\left[\left(f(x_0) + f(x_n)\right) + 2(f(x_1) + f(x_2) + \cdots + f(x_{n-1}))\right]$$

$$\textit{Toplam Trapezoid Alanları} = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

Seri Program Pseudo-Code

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 0; i <= n-1; i++) {
    x_i = a + i*h;
    approx += f(x_i);
}
approx = h*approx;
```

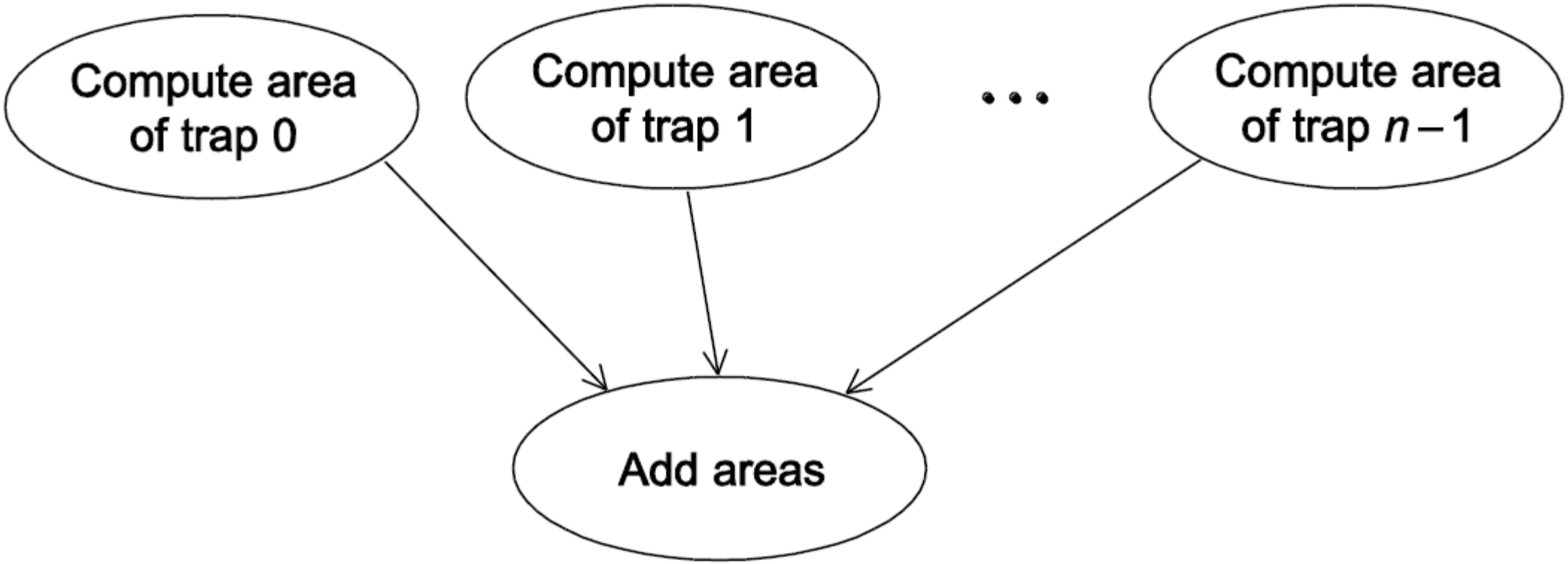
Trapezoidal Kuralının Paralelleştirilmesi

- Problemi çözmek için küçük parçalara ayırın.
- Görevler (tasks) arasındaki iletişim kanallarını tanımlayın.
- Görevleri bileşik görev üzerinde toplayın (Agrregation Tasks).
- Bileşik görevleri işleme eşleyin.

Trapezoidal Kuralının Paralel Pseudo-Code Yapısı

```
Get a, b, n;
h = (b−a)/n;
local_n = n/comm_sz;
local_a = a + my_rank*local_n*h;
local_b = local_a + local_n*h;
local_integral = Trap(local_a, local_b, local_n, h);
if (my_rank != 0)
   Send local_integral to process 0;
else /* my_rank == 0 */
   total_integral = local_integral;
   for (proc = 1; proc < comm_sz; proc++) {
      Receive local_integral from proc;
      total_integral += local_integral;
   }
}
if (my_rank == 0)
   print result;
```

Paralel Trapez Kuralı için Görev ve İletişim Tanımları



Paralel Trepozoid C Kodu *1

```
int main(void) {
   int my_rank, comm_sz, n = 1024, local_n;
   double a = 0.0, b = 3.0, h, local_a, local_b;
   double local_int, total_int;
   int source;

   MPI_Init(NULL, NULL);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

   h = (b-a)/n;          /* h is the same for all processes */
   local_n = n/comm_sz;  /* So is the number of trapezoids  */

   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   local_int = Trap(local_a, local_b, local_n, h);

   if (my_rank != 0) {
      MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
            MPI_COMM_WORLD);
```

Paralel Trepozoid C Kodu *2

```
    } else {
        total_int = local_int;
        for (source = 1; source < comm_sz; source++) {
            MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total_int += local_int;
        }
    }

    if (my_rank == 0) {
        printf("With n = %d trapezoids, our estimate\n", n);
        printf("of the integral from %f to %f = %.15e\n",
            a, b, total_int);
    }
    MPI_Finalize();
    return 0;
} /* main */
```

Paralel Trepozoid C Kodu *3

```
double Trap(
        double left_endpt  /* in */,
        double right_endpt /* in */,
        int    trap_count  /* in */,
        double base_len    /* in */) {
    double estimate, x;
    int i;

    estimate = (f(left_endpt) + f(right_endpt))/2.0;
    for (i = 1; i <= trap_count-1; i++) {
        x = left_endpt + i*base_len;
        estimate += f(x);
    }
    estimate = estimate*base_len;

    return estimate;
} /* Trap */
```

MPI I/O Kontrolü

```
#include <stdio.h>
#include <mpi.h>

int main(void) {
   int my_rank, comm_sz;

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   printf("Proc %d of %d > Does anyone have a toothpick?\n",
         my_rank, comm_sz);

   MPI_Finalize();
   return 0;
}  /* main */
```

Her bir İşlem ekrana bir Mesaj yazıyor

MPI I/O Kontrolü

6

```
Proc 0 of 6 > Does anyone have a toothpick?
Proc 1 of 6 > Does anyone have a toothpick?
Proc 2 of 6 > Does anyone have a toothpick?
Proc 4 of 6 > Does anyone have a toothpick?
Proc 3 of 6 > Does anyone have a toothpick?
Proc 5 of 6 > Does anyone have a toothpick?
```

Hangi işlemin daha önce çalışacağını kestiremiyoruz.

Bunu durumu bir sıraya sokmamız gerekiyor.

MPI ile Kullanıcıdan Girdi Almak

- Çoğu MPI uygulaması yalnızca 0 işlemine MPI_COMM_WORLD erişimi izni verir.
- İşlem 0 verileri (scanf) okumak ve diğer processlere göndermekle sorumludur.
- Bu işlemi kullanabilmek için main fonksiyonu içerisinde yeni bir fonksiyon çağırılması yeterli olacaktır.

```
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &p);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

Get_input(my_rank,comm_sz, &a, &b, &n);
```

Kullanıcı Girdisi Okuma Fonksiyonu

```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MPI ile Hesaplama Süresi Denetimi

- Çoğu paralel programda sistemin çalışma sürelerini tutmak önemlidir.
- Bu süreleri hesaplayabilmek için MPI_Wtime() fonksiyonunu kullanarak aradan geçen süre hesaplanabilir.

```
double start, finish;
. . .
start = MPI_Wtime();
/* Code to be timed */
. . .
finish = MPI_Wtime();
printf("Proc %d > Elapsed time = %e seconds\n"
      my_rank, finish-start);
```

Seri olarak Hesaplama Süresi Denetimi

- Bazı durumlarda GET_TIME(double) fonksiyonunda kullanabilirsiniz.

```
#include "timer.h"
. . .
double now;
. . .
GET_TIME(now);
```

```
#include "timer.h"
. . .
double start, finish;
. . .
GET_TIME(start);
/* Code to be timed */
. . .
GET_TIME(finish);
printf("Elapsed time = %e seconds\n", finish-start);
```

Teşekkürler.

Sorular...